

# РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

## ОТЧЕТ

### ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8

дисциплина:     Архитектура компьютера

Студент: Савенкова Татьяна Александровна

Группа: НКАбд-05-25

МОСКВА

2025 г.

## Оглавление

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы.....</b>                     | <b>4</b>  |
| <b>2</b> | <b>Задание.....</b>                         | <b>5</b>  |
| 4.1      | Реализация циклов в NASM .....              | 7         |
| 4.2      | Обработка аргументов командной строки ..... | 9         |
| 4.3      | Задание для самостоятельной работы .....    | 12        |
| <b>5</b> | <b>Выводы .....</b>                         | <b>14</b> |
| <b>6</b> | <b>Список литературы.....</b>               | <b>15</b> |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Рисунок 1 Создание каталога .....                               | 7  |
| Рисунок 2 Копирование программы из листинга .....               | 7  |
| Рисунок 3 Запуск программы.....                                 | 8  |
| Рисунок 4 Изменение программы .....                             | 8  |
| Рисунок 5 Запуск измененной программы .....                     | 9  |
| Рисунок 6 Копирование программы из листинга .....               | 9  |
| Рисунок 7 Запуск второй программы .....                         | 10 |
| Рисунок 8 Копирование программы из третьего листинга.....       | 10 |
| Рисунок 9 Запуск третьей программы .....                        | 11 |
| Рисунок 10 Изменение третьей программы .....                    | 11 |
| Рисунок 11 Запуск измененной третьей программы .....            | 11 |
| Рисунок 12 Написание программы для самостоятельной работы ..... | 12 |
| Рисунок 13 Запуск программы для самостоятельной работы .....    | 13 |

## **1    Цель работы**

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

## **2   Задание**

Реализация циклом в NASM

Обработка аргументов командной строки

Самостоятельное написание программы по материалам лабораторной работы

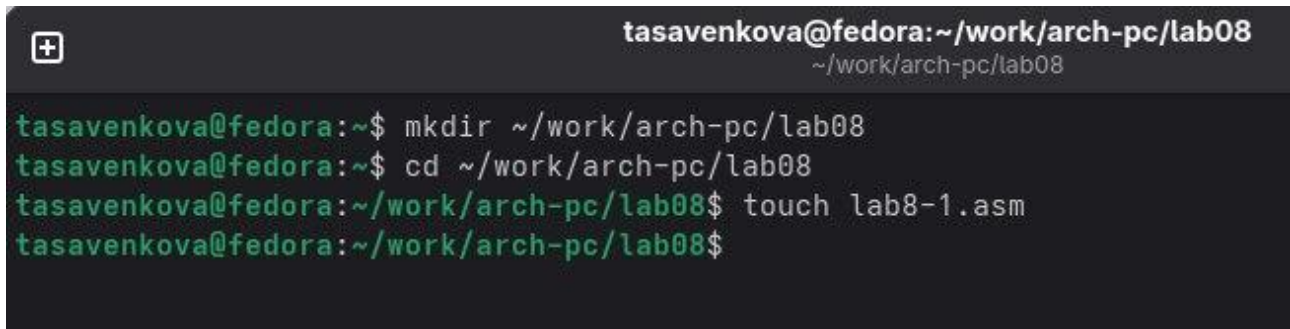
### **3 Теоретическое введение**

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out» или «последним пришёл — первым ушёл»). Стек является частью архитектуры процессора и реализован на аппаратном уровне. Для работы со стеком в процессоре есть специальные регистры (ss, bp, sp) и команды. Основной функцией стека является функция сохранения адресов возврата и передачи аргументов при вызове процедур. Кроме того, в нём выделяется память для локальных переменных и могут временно храниться значения регистров.

## 4 Выполнение лабораторной работы

### 4.1 Реализация циклов в NASM

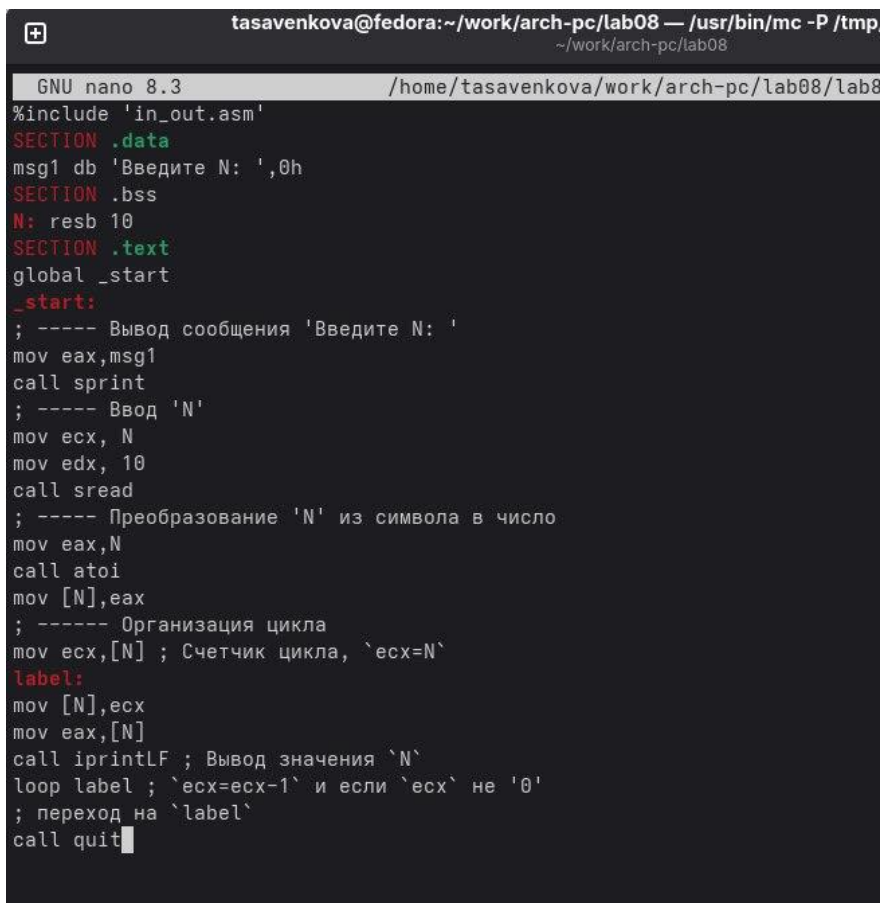
Создаю каталог для программ лабораторной работы №8 (Рисунок 1).



```
tasavenkova@fedora:~/work/arch-pc/lab08
tasavenkova@fedora:~$ mkdir ~/work/arch-pc/lab08
tasavenkova@fedora:~$ cd ~/work/arch-pc/lab08
tasavenkova@fedora:~/work/arch-pc/lab08$ touch lab8-1.asm
tasavenkova@fedora:~/work/arch-pc/lab08$
```

Рисунок 1 Создание каталога

Копирую в созданный файл программу из листинга. (Рисунок 2).



```
GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab08/lab8
%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
mov [N],ecx
mov eax,[N]
call iprintLF ; Вывод значения `N`
loop label ; `ecx=ecx-1` и если `ecx` не `0`
; переход на `label`
call quit
```

Рисунок 2 Копирование программы из листинга

Запускаю программу, она показывает работу циклов в NASM (Рисунок 3).

```

tasavenkova@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
tasavenkova@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 6
6
5
4
3
2
1
tasavenkova@fedora:~/work/arch-pc/lab08$

```

Рисунок 3 Запуск программы

Заменяю программу изначальную так, что в теле цикла я изменяю значение регистра `ecx` (Рисунок 4).

```

GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab08/lab8-1.asm
#include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
push ecx ; добавление значения ecx в стек
sub ecx,1
mov [N],ecx
mov eax,[N]
call iprintLF
pop ecx ; извлечение значения ecx из стека
loop label

call quit

```

Рисунок 4 Изменение программы

Из-за того, что теперь регистр `ecx` на каждой итерации уменьшается на 2 значения, количество итераций уменьшается вдвое (Рисунок 5).



```

tasavenkova@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
tasavenkova@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 6
5
4
3
2
1
0
tasavenkova@fedora:~/work/arch-pc/lab08$

```

Рисунок 5 Запуск измененной программы

## 4.2 *Обработка аргументов командной строки*

Создаю новый файл для программы и копирую в него код из следующего листинга (Рисунок 6).

```

GNU nano 8.3 /home/tasavenkova/wo
#include 'in_out.asm'
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
              ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
              ; (второе значение в стеке)
    sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
              ; аргументов без названия программы)
next:
    cmp ecx, 0 ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
              ; (переход на метку `_end`)
    pop eax ; иначе извлекаем аргумент из стека
    call sprintf ; вызываем функцию печати
    loop next ; переход к обработке следующего
              ; аргумента (переход на метку `next`)
_end:
    call quit

```

Рисунок 6 Копирование программы из листинга

Компилирую программу и запускаю, указав аргументы. Программой было обработано то же количество аргументов, что и было введено (Рисунок 7).

```

tasavenkova@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
tasavenkova@fedora:~/work/arch-pc/lab08$ ./lab8-2 arg1 arg 2 'arg 3'
arg1
arg
2
arg 3
tasavenkova@fedora:~/work/arch-pc/lab08$

```

Рисунок 7 Запуск второй программы

Создаю новый файл для программы и копирую в него код из третьего листинга (Рисунок 8).

```

GNU nano 8.3 /home/tasavenkova/work/a
#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
              ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
              ; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
              ; аргументов без названия программы)
    mov esi, 0 ; Используем `esi` для хранения
              ; промежуточных сумм
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
              ; (переход на метку `_end`)
    pop eax ; иначе извлекаем следующий аргумент из стека
    call atoi ; преобразуем символ в число
    add esi,eax ; добавляем к промежуточной сумме
              ; след. аргумент `esi=esi+eax`
    loop next ; переход к обработке следующего аргумента
_end:
    mov eax, msg ; вывод сообщения "Результат: "
    call sprint
    mov eax, esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
    call quit

```

Рисунок 8 Копирование программы из третьего листинга

Компилирую программу и запускаю, указав в качестве аргументов некоторые числа, программа их складывает (Рисунок 9).

```

tasavenkova@fedora:~/work/arch-pc/lab08$ touch lab8-3.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
tasavenkova@fedora:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 47
tasavenkova@fedora:~/work/arch-pc/lab08$

```

Рисунок 9 Запуск третьей программы

Изменяю поведение программы так, чтобы указанные аргументы она умножала, а не складывала (Рисунок 10).

```

GNU nano 8.3 /home/tasavenkova/work/
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
por ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
por edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
mov esi, 1 ; Используем `esi` для хранения
; промежуточных сумм
next:
cmp ecx,0h ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
por eax ; иначе извлекаем следующий аргумент из стека
call atoi ; преобразуем символ в число
mul esi
mov esi,eax

loop next ; переход к обработке следующего аргумента
_end:
mov eax, msg ; вывод сообщения "Результат: "
call sprint
mov eax, esi ; записываем сумму в регистр `eax`
call iprintLF ; печать результата
call quit

```

Рисунок 10 Изменение третьей программы

Программа действительно теперь умножает данные на вход числа (Рисунок 11).

```

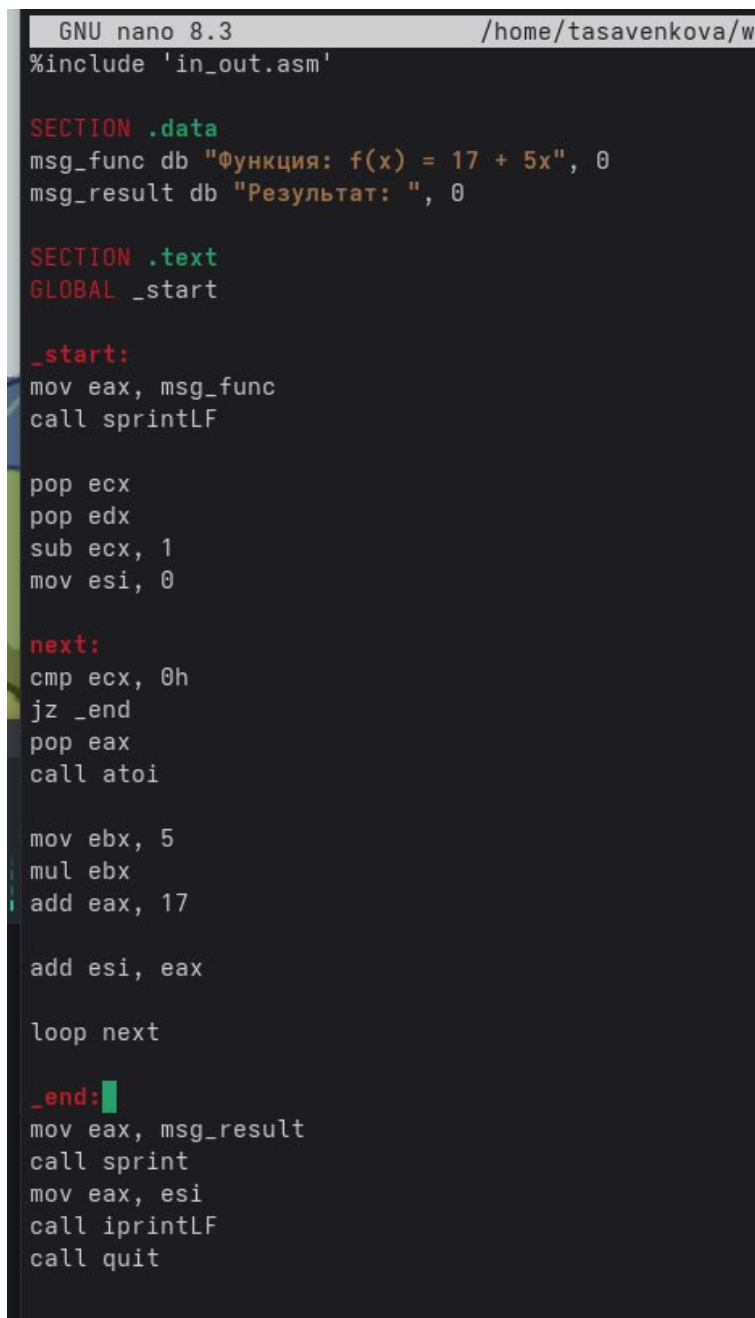
tasavenkova@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
tasavenkova@fedora:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 54600
tasavenkova@fedora:~/work/arch-pc/lab08$

```

Рисунок 11 Запуск измененной третьей программы

### 4.3 Задание для самостоятельной работы

Пишу программу, которая будет находить сумму значений для функции  $f(x) = 17 + 5x$ , которая совпадает с моим 18 вариантом (Рисунок 12).



```
GNU nano 8.3 /home/tasavenkova/w
#include 'in_out.asm'

SECTION .data
msg_func db "Функция: f(x) = 17 + 5x", 0
msg_result db "Результат: ", 0

SECTION .text
GLOBAL _start

_start:
mov eax, msg_func
call sprintf

pop ecx
pop edx
sub ecx, 1
mov esi, 0

next:
cmp ecx, 0h
jz _end
pop eax
call atoi

mov ebx, 5
mul ebx
add eax, 17

add esi, eax

loop next

_end:
mov eax, msg_result
call sprintf
mov eax, esi
call iprintLF
call quit
```

Рисунок 12 Написание программы для самостоятельной работы

Проверяю работу программы, указав в качестве аргумента несколько чисел (Рисунок 13).

```
tasavenkova@fedora:~/work/arch-pc/lab08$ touch lab8-4.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-4.asm
tasavenkova@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-4 lab8-4.o
tasavenkova@fedora:~/work/arch-pc/lab08$ ./lab8-4 1 2 3 4
Функция:  $f(x) = 17 + 5x$ 
Результат: 118
tasavenkova@fedora:~/work/arch-pc/lab08$
```

Рисунок 13 Запуск программы для самостоятельной работы

## **5 Выводы**

В результате выполнения данной лабораторной работы я приобрел навыки написания программ с использованием циклов а также научился обрабатывать аргументы командной строки.

## **6      Список литературы**

1. Курс на ТУИС
2. Лабораторная работа №8
3. Программирование на языке ассемблера NASM Столяров А. В.